

II. S.O.L.I.D. Principles

1. What is the contract of the following method?

```
public static Collection<String> readLinesFromFile(String absoluteFilePath)
```

Preconditions:

Postconditions:

Invariants:

Side effects:

2. What principle is violated in the following example? Explain your answer. How would you refactor the code to not violate the principle?

```
interface IMembership {  
    void login(UserCredentials uc);  
    boolean logout();  
    UserCredentials register(String userName, String password);  
    UserCredentials changePassword(String userName);  
    void addPaymentMethod(UserCredentials uc, PaymentMethod pm);  
}
```

3. Does the following method respect design by contract? Why? How can you ensure that the base method's contract will never be breached by a method that is overriding it?

```
String Base :: aMethod(int[] args)  
//PRECONDITIONS: args contains only negative numbers  
//POSTCONDITIONS: returns a String with only lowercase characters or  
digits
```



```
String Derived :: aMethod(int[] args)  
//PRECONDITIONS: args contains any negative or even numbers  
//POSTCONDITIONS: returns a String composed only of digits or null
```

4. Imagine you have a class **ProductsOffer** that stores a collection of **Products**. Each product has name, manufacturer name, color, price, weight and release date. In the first version **ProductsOffer** provides the next services for filtering the products:

```
getByName()
getByManufacturerName()
getByColor()
getByPrice()
getByWeight()
getByReleaseDate()
```

After a while the customer needs to filter by name and color so we add the method: `getByNameAndColor()`

After another while the customer needs to filter by weight and date so we add the method: `getByWeightAndReleaseDate()`

Is the class designed well? What principle is violated? Explain why. How should you refactor the class so that it easily fits the future needs of the customer? Implement the code.

5. What principle is violated in the following example? Explain why. How would you refactor the code to not violate the principle?

```
abstract class Vehicle {
    private int peopleCapacity;

    abstract double getSpeed();
    abstract double getGasLevel();
    abstract void pressClutch();
}

class Scooter extends Vehicle{
    //does not have a clutch
}

class Tesla extends Vehicle{
    //Tesla is an Electric Car
    int getTrunkCapacity(){...}
    int getFrunkCapacity(){...}
    double getAutonomy(){...}
}

class Logan extends Vehicle{
    int getTrunkCapacity(){...}
}
```

6. What principle is violated in the following example? Explain why. How would you refactor the code? Draw a class diagram showing your solution.

```
class Student {
    ...
    public void saveStudent();
    public Double calculateMeanGrade();
    public Profile getStudentProfile();
}
```